2.5.1 An example of a polymorphic function

We can often discover polymorphic functions by observing that several monomorphic functions all share a similar structure. For example, the following monomorphic function, `findFirst`, returns the first index in an array where the key occurs, or `-1` if it's not found. It's specialized for searching for a `String` in an `Array` of `String` values.

Listing 2.3 Monomorphic function to find a `String` in an array

```
def findFirst(ss: Array[String], key: String): Int = {
  @annotation.tailrec
  def loop(n: Int): Int =
    if (n >= ss.length) -1
    else if (ss(n) == key) n
    else loop(n + 1)

  loop(0)
}
```

Otherwise, increment `n` and keep looking.

If `n` is past the end of the array, return `-1`, indicating the key doesn't exist in the array.

`ss(n)` extracts the nth element of the array `ss`. If the element at `n` is equal to the key, return `n`, indicating that the element appears in the array at that index.

Start the loop at the first element of the array.

The details of the code aren't too important here. What's important is that the code for `findFirst` will look almost identical if we're searching for a `String` in an `Array[String]`, an `Int` in an `Array[Int]`, or an `A` in an `Array[A]` for any given type `A`. We can write `findFirst` more generally for any type `A` by accepting a function to use for testing a particular `A` value.

Listing 2.4 Polymorphic function to find an element in an array

```
def findFirst[A](as: Array[A], p: A => Boolean): Int = {
  @annotation.tailrec
  def loop(n: Int): Int =
    if (n >= as.length) -1
    else if (p(as(n))) n
    else loop(n + 1)

  loop(0)
}
```

Instead of hardcoding `String`, take a type `A` as a parameter. And instead of hardcoding an equality check for a given key, take a function with which to test each element of the array.

If the function `p` matches the current element, we've found a match and we return its index in the array.

This is an example of a polymorphic function, sometimes called a *generic* function. We're *abstracting over the type* of the array and the function used for searching it. To write a polymorphic function as a method, we introduce a comma-separated list of *type parameters*, surrounded by square brackets (here, just a single `[A]`), following the name of the function, in this case `findFirst`. We can call the type parameters anything we want—`[Foo, Bar, Baz]` and `[TheParameter, another_good_one]` are valid type parameter declarations—though by convention we typically use short, one-letter, uppercase type parameter names like `[A,B,C]`.